

VQ-VAE复现

汇报人：江经纬

问题与贡献

VQ-VAE解决的问题

- 传统VAE面临“后验坍缩”问题：
 - 解码器能力过强，绕过潜在表示，直接依靠自身的能力建模数据分布。
 - 此时潜在变量 z 不携带有效信息，退化为无意义信息。
- 问题原因
 - 传统VAE采用连续隐变量空间，模型可以“偷懒”让后验隐空间分布直接等于先验隐空间分布，使KL散度项直接为0

VQ-VAE的贡献

- 1. 使用离散隐变量表示，直接避免了KL散度项为0导致的后验坍缩。
- 2. 性能上能够媲美采取连续隐空间的传统VAE模型。
- 3. 应用场景广阔，可以拓展应用于音频、视频生成等领域。

方法

方法

- 采取离散隐变量表示：
 - 1. 建立一个码本空间 $e = \text{Embedding}(K, D)$
 - 2. 对于encoder给出的一条特征 $z_e(x)$ ，假定形状为 $(1, D)$ ，通过类似最近邻算法得到其离散隐变量表示

$$q(z = k|x) = \begin{cases} 1 & \text{for } k = \operatorname{argmin}_j \|z_e(x) - e_j\|_2, \\ 0 & \text{otherwise} \end{cases},$$

- 3. 对于得到的离散隐变量表示，回到码本空间中查询该索引对应的特征 $z_q(x)$ ，将对应的特征送入decoder

$$z_q(x) = e_k, \quad \text{where } k = \operatorname{argmin}_j \|z_e(x) - e_j\|_2$$

离散隐变量的生成过程

encoder给出特征
 $z_e(x)=[1.0, 2.0, 3.0]$

码本空间e

$e_1=[5.0, 6.3, 9.4]$

$e_2=[1.2, 2.1, 3.4]$

$e_3=[-1.7, 5.2, 10.2]$

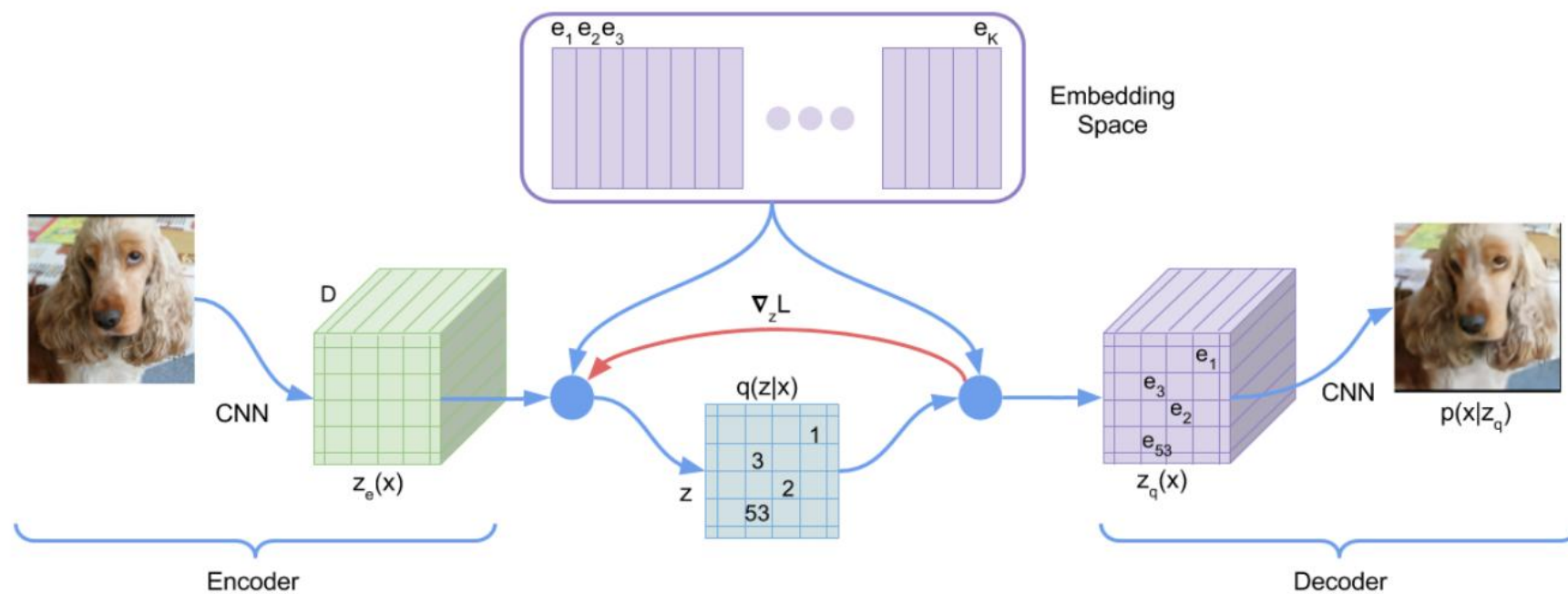
$e_4=[-2.9, 9.4, 3.0]$

$z_e(x)$ 与 e_3 最接近,
因此其对应的离散
隐变量表示就是3

对于得到的离散隐
变量, 从码本中查
询得到 $z_q(x)=e_3$,
将 $z_q(x)$ 送入
decoder

模型架构

模型架构



Encoder

- input: (B, C, H, W)
- 2个步长为2的 $4*4$ 卷积层
- (四倍下采样)
- 2个残差块($\text{relu}+3*3\text{conv}+\text{relu}+1*1\text{conv}$)
- output: (B, D, H, W)

Encoder: Residual

```
class Residual(nn.Module):
    def __init__(self, in_channels, residual_channels,
out_channels):
        super().__init__()
        self.block=nn.Sequential(
            nn.ReLU(True),
            nn.Conv2d(in_channels, residual_channels,
kernel_size=3, stride=1, padding=1),
            nn.ReLU(True),
            nn.Conv2d(residual_channels, out_channels,
kernel_size=1, stride=1)
        )
    def forward(self, x):
        return x+self.block(x)
```

单个残差块：
relu+减少通道的卷积+relu+恢复通道的卷积

```
class ResidualStack(nn.Module):
    def __init__(self, in_channels, residual_channels,
out_channels, residual_layers):
        super().__init__()
        self._residual_layers=residual_layers
        self._layers=nn.ModuleList(
            [Residual(in_channels, residual_channels,
out_channels)
            for _ in range(self._residual_layers)]
        )
    def forward(self, x):
        for i in range(self._residual_layers):
            x=self._layers[i](x)
        return F.relu(x)
```

残差块堆积

Encoder：完整模块

结构

conv1: 4*4, stride=2, padding=1

conv2: 4*4, stride=2, padding=1

对原图做4倍下采样

conv3: 3*3, stride=1, padding=1

对特征做预处理，送入残差模块

residualstack: 堆积两个残差块，对特征做进一步处理

```
class Encoder(nn.Module):
    def __init__(
        self, in_channels, hidden_channels, residual_channels,
        residual_layers):
        super().__init__()
        # (in_c, 32, 32) -> (hid_c//2, 16, 16)
        self.conv1 = nn.Conv2d(
            in_channels, hidden_channels//2,
            kernel_size=4, stride=2, padding=1
        )
        # (hid_c//2, 16, 16) -> (hid_c, 8, 8)
        self.conv2 = nn.Conv2d(
            hidden_channels//2, hidden_channels,
            kernel_size=4, stride=2, padding=1
        )
        # (hid_c, 8, 8) -> (hid_c, 8, 8)
        # 第三层卷积对特征做一个处理再去接relu
        self.conv3 = nn.Conv2d(
            hidden_channels, hidden_channels,
            kernel_size=3, stride=1, padding=1
        )
        # (hid_c, 8, 8) -> (hid_c, 8, 8)
        self.residual_stack = ResidualStack(hidden_channels,
            residual_channels, hidden_channels, residual_layers)
    def forward(self, x):
        out = F.relu(self.conv1(x))
        out = F.relu(self.conv2(out))
        out = self.conv3(out)
        return self.residual_stack(out)
```

Decoder

- input: (B, D, H, W)
- 2个残差块
- 2个卷积层做4倍上采样
- output: (B, C, H, W)

Decoder: 完整结构

结构:

conv1: 3×3 , stride=1, padding=1
对输入特征做一个预处理, 将通道维度变换为decoder的对应维度

residualstack: 堆积两个残差模块,
对输入特征进行处理

conv2: 4×4 , stride=2, padding=1

conv3: 4×4 , stride=2, padding=1
上采样变回原图

```
class Decoder(nn.Module):
    def __init__(self, in_channels, hidden_channels,
                  out_channels, residual_channels, residual_layers):
        super().__init__()
        self.conv1 = nn.Conv2d(in_channels, hidden_channels,
                                kernel_size=3, stride=1, padding=1)

        self.residual_stack = ResidualStack(hidden_channels,
                                              residual_channels, hidden_channels, residual_layers)

        self.conv2 = nn.ConvTranspose2d(hidden_channels,
                                          hidden_channels // 2, kernel_size=4, stride=2, padding=1)

        self.conv3 = nn.ConvTranspose2d(hidden_channels // 2,
                                          out_channels, kernel_size=4, stride=2, padding=1)
    def forward(self, x):
        out = self.conv1(x)
        out = self.residual_stack(out)
        out = F.relu(self.conv2(out))
        return self.conv3(out)
```

VectorQuantization